

MEMO-Appflow

从 Android OS 证据到端侧 MAPLE 推理
再回到应用推荐与系统调度

讲解人: Jingyi Guo

组员: Mosha Huang Kewei Chen Chengyu Fan

rooted Pixel 5 | raw eBPF/libbpf | local MAPLE

Top-3 真实应用推荐 + scheduler bits 系统调度

汇报路线

- 1 从 app + OS 到 app + OS
- 2 eBPF 与 Kernel 采集层
- 3 数据结构化管线
- 4 MAPLE 推理
- 5 预测之后：真实 app 与系统调度
- 6 真实实验
- 7 实现文件与结论

项目不是单纯的 app 推荐

用户看到的现象

- 刚打开聊天软件，马上想拍照或分享。
- 正在浏览网页，网络和渲染压力升高。
- 手机缓存很多 app 后，启动等待变长。

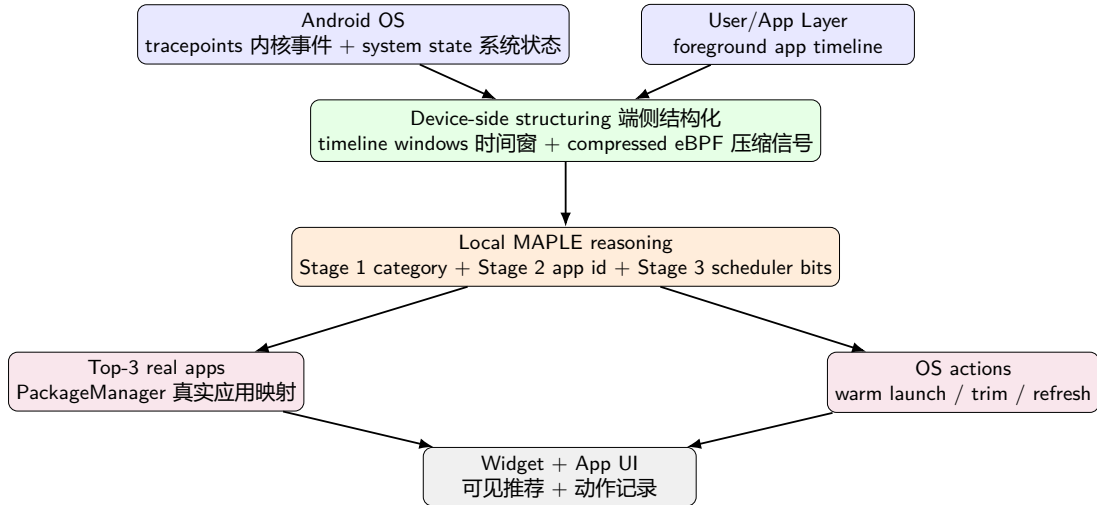
OS 真实承受的压力

- Binder 服务调用、前台切换、进程 exec。
- UDP send/recv、文件 open、runtime loading。
- vmscan reclaim、SurfaceFlinger、调度事件。

MEMO-Appflow 的核心判断

只看 app 序列不够，只看内核事件也不够。我们把 app 行为和 OS 证据合并成端侧模型上下文，再让模型同时输出用户接下来可能用的 app 和系统应该执行的调度动作。

宏观闭环：系统观察、模型决策、系统执行



MAPLE 是本项目本地大模型推理模块；scheduler bits 是模型输出的一串 0/1 系统动作开关；Widget 是桌面小组件，用来展示可点击的 Top-3 app。这条链路在手机内部运行，host 只负责安装、推模型、拉报告。

手机内运行边界

Phone-local (手机本地执行)

- raw eBPF/libbpf collector: 原生内核事件采集器
- foreground app sampler
- system state collector: 内存、网络、电量等状态采集器
- Kotlin parser and scenario builder: Kotlin 解析与场景构造
- local MAPLE native inference: 本地 MAPLE 推理
- Top-3 mapping and ActionExecutor: Top-3 映射与动作执行
- desktop Widget update: 桌面小组件刷新

Host only

- install APK
- push MAPLE binary and GGUF model
- trigger experiment during debugging
- pull reports for inspection
- compile LaTeX / write paper

设计原则

产品逻辑不能依赖电脑脚本；拔线后手机仍能通过 App 和 Widget 运行核心闭环。

为什么要引入 eBPF

eBPF 是 Linux/Android 内核里的安全观测机制。我们用它在内核事件发生时记录证据，而不是只看 app 层日志。

App 序列能告诉我们

- 用户刚打开了哪个应用。
- 应用切换的大致顺序。
- 最近是否处于社交、浏览、媒体等工作流。

eBPF 能补上

- 网络收发是否活跃：sendto/recvfrom。
- 服务调用是否密集：Binder transaction。
- 内存是否紧张：direct reclaim / kswapd。
- UI 是否繁忙：sched、input、SurfaceFlinger hints。

App sequence describes intent; eBPF describes the hidden OS work behind that intent.

采集的 14 个 tracepoints

Tracepoint 是内核预留的稳定事件点，例如系统调用进入、Binder transaction、进程 exec。collector attach 到这些点上收真实 OS 证据。

Group	Tracepoint	用途
IPC	tracepoint/binder/binder_transaction	Android service / Binder pressure
File	syscalls/sys_enter_openat	file I/O, runtime loading
File	syscalls/sys_enter_openat2	file I/O, camera/media file hints
Network	syscalls/sys_enter_sendto	network send, especially UDP
Network	syscalls/sys_enter_recvfrom	network receive, especially UDP
Memory	vmscan/mm_vmscan_direct_reclaim_begin	direct reclaim starts
Memory	vmscan/mm_vmscan_direct_reclaim_end	direct reclaim ends
Memory	vmscan/mm_vmscan_kswapd_wake	kswapd wakeup
Scheduling	sched/sched_switch	CPU scheduling activity
Scheduling	sched/sched_wakeup	wakeup pressure
Process	sched/sched_process_fork	process lifecycle
Process	sched/sched_process_exit	process lifecycle
Process	sched/sched_process_exec	process launch
Input	input/input_event	touch/key event

实现位置

raw BPF object (BPF 字节码) 和 collector (采集器) 部署到 /data/local/tmp/memo; Android 端通过 root shell 启动。

Pixel 5 上的 kernel/runtime 条件

Device

- Google Pixel 5
- arm64-v8a
- Magisk root
- custom Android kernel path
- SELinux relaxed for tracing/debug deployment

Kernel features

- CONFIG_BPF
- CONFIG_BPF_SYSCALL
- CONFIG_BPF_JIT
- CONFIG_FTRACE
- CONFIG_FTRACE_SYSCALLS
- CONFIG_KPROBES, CONFIG_UPROBES

工程选择

bpfftrace 曾用于探索，但最终产品路径以 raw eBPF/libbpf collector 为主，减少运行时依赖，也更适合 Android 端部署。

tracepoint group	counter key / type
binder transaction	1 / binder
openat, openat2	2 / file
vmscan begin/end/kswapd	3 / memory
sendto	4 / network-send
recvfrom	5 / network-recv
sched switch/wakeup	6 / sched
process fork	7 / process-fork
process exit	8 / process-exit
process exec	9 / process-exec
input event	10 / input

左边每一行表示一个内核 tracepoint 族会被归成哪一种计数键。比如 sendto/recvfrom 都是网络收发证据，计数后变成 network-send/network-recv。这样做不是丢掉 eBPF，而是把重复低层事件压成模型能读懂的“某类系统活动发生了多少次、每秒多密集”。

为什么先计数

- eBPF 原始行可能上万条。
- 大量重复事件直接进 prompt 会浪费上下文长度。
- counter 保存 count/rate，保留“强度”和“密度”。
- raw trace 仍保留在报告目录用于复查。

真实采集窗口长什么样

来自 30-record 实验的窗口摘要

```
raw_trace_path:
/sdcard/MEMO/logs/realtime_1782914935996.trace

app_samples_path:
/sdcard/MEMO/logs/realtime_1782914935996_apps.tsv

raw_ebpf_events: 13733
app_segments: 6
timeline_windows: 2
```

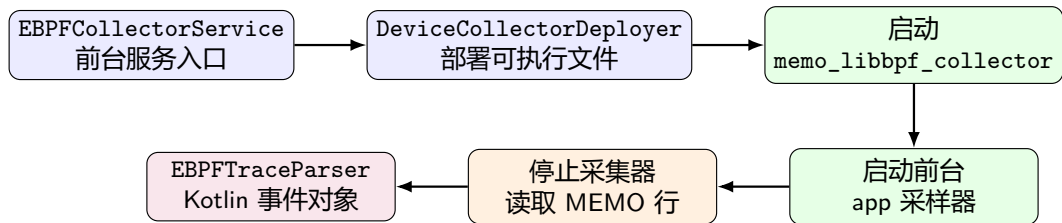
进入 MAPLE 前会被压缩

```
{
  "event_type": "MEMO_RECVFROM",
  "detail": "recvfrom",
  "count": 258462,
  "rate_per_sec": 1367.2
}
```

这一页展示的是一次手机端真实采集窗口：raw trace 是原始 eBPF 文件，app samples 是同一时间内前台 app 记录。右边的 count 和 rate_per_sec 表示同类事件在窗口里重复出现的次数和频率，避免把几十万条相似网络收包逐行塞进 prompt。

Source: docs/real_device_experiments/synthetic_user_30_2026_07_01

Android 端采集运行时：代码结构概览



这一页是

在说明“一次采集窗口”在 Android 里怎么跑完: EBPFCollectorService 接收按钮、广播或实时任务入口; DeviceCollectorDeployer 确保手机本地有可执行的 raw eBPF collector; memo_libbpf_collector 在 root 环境 attach tracepoint 并写出 MEMO_* 行; 前台 app 采样器同步记录当前可见 app; 窗口结束后停止 collector, 读取 trace 文件; EBPFTraceParser 把文本行转成 Kotlin 事件对象, 交给后面的 timeline/window 压缩。

Source: `app/src/main/java/com/memoos/realtime/RealtimeWindowRunner.kt`

app 序列和 eBPF 的时间对齐

Foreground app sampler (前台 app 采样器)

每秒读取:

- topResumedActivity
- ResumedActivity
- mCurrentFocus
- mFocusedApp

然后合并成 app segment。

eBPF event alignment (内核事件时间对齐)

- 有 wall time: 直接对齐。
- 无 wall time: 按窗口内 index 估计。
- 每个 timeline window 绑定 app labels 和 compressed eBPF。

必须对齐是因为同一条 eBPF 证据在不同 app 场景下含义不同: Chrome 前台时大量 `recvfrom` 更像浏览/视频网络流, Camera 前台时文件和显示事件更像拍照链路。对齐后, MAPLE 看到的不只是“网络很多”, 而是“哪个时间段、哪个 app 前台时、哪些 OS 信号一起出现”。

timeline window 示例

```
{
  "index": 1,
  "start_time": "2026-07-01T22:09:23+08:00",
  "end_time": "2026-07-01T22:09:37+08:00",
  "app_labels": [
    "Clock",
    "Google Play Movies & TV"
  ],
  "app_packages": [
    "com.google.android.deskclock",
    "com.google.android.videos"
  ],
  "compressed_ebpf": [
    {"event_type": "MEMO_RECVFROM", "count": 258462, "rate_per_sec": 1367.2},
    {"event_type": "MEMO_BINDER", "count": 134183, "rate_per_sec": 709.8}
  ]
}
```

这个 window 表示 14 秒左右的真实手机时间片：上半部分是这段时间看到的 app，下面是同一段时间压缩后的 eBPF 证据。模型可以据此判断“用户正在从 Clock 切换到视频/媒体相关 app，同时网络收包和 Binder 通信很密集”。

eventType 到 category: 权重矩阵

eventType	Network	Binder	Runtime	File	Memory	Input
MEMO_SENDTO	3	0	0	0	0	0
MEMO_RECVFROM	3	0	0	0	0	0
MEMO_BINDER	0	2	0	0	0	0
MEMO_SCHED	0	0	1	0	0	0
MEMO_PROCESS_FORK	0	0	1	0	0	0
MEMO_PROCESS_EXIT	0	0	1	0	0	0
MEMO_PROCESS_EXEC	0	0	1	0	0	0
MEMO_OPENAT	0	0	0	1	0	0
MEMO_MEMORY	0	0	0	0	8	0
MEMO_INPUT	0	0	0	0	0	4
cap	420	420	180	120	360	420

行是采集到的 eBPF event type, 列是给

MAPLE 的语义类别。数字表示这个事件对该类别加多少分; 例如一次 MEMO_RECVFROM 给 Network 加 3 分, 一次 MEMO_MEMORY 给 Memory 加 8 分。最后一行 cap 是上限, 防止网络这种高频事件把 Binder、Input、Memory 等低频但重要的信号淹没。

Category score 如何算

例：Network IO

```
raw counter:
    MEMO_SENDTO    count = 120
    MEMO_RECVFROM  count = 180

weight:
    SENDTO -> Network +3
    RECVFROM -> Network +3

raw score = 120*3 + 180*3 = 900
cap(Network) = 420
Network IO score = min(900, 420)
```

为什么要 cap

Network events are high frequency.
If not capped:
network may dominate all categories.

After cap:
Network still means "very active",
but Binder/Memory/Input can still matter.

Category score 是“语义强度分”：先数每类 eBPF 事件出现多少次，再乘以权重矩阵里的分数，最后用 cap 截断。它不是概率，也不是最终预测；它只是告诉 MAPLE 最近窗口里哪些系统活动最强。关键修正：Memory Management、Android Service IPC、App Process Runtime 不是 app。它们能说明“内存紧张 / Binder 忙 / 进程切换多”，所以影响调度动作；但不能显示成“下一个 app”。因此 context 会分开保存 app 候选和 OS 调度证据。

Context JSON 字段

Context JSON 是 MAPLE 每次推理真正读取的输入文件：它把“刚才用了哪些 app”和“同一时间 OS 发生了什么”压成一个结构化对象。

Field	Meaning
historical_app_categories	用户可见 app 预测候选
scheduler_evidence_categories	OS/eBPF 调度证据类别
historical_app_ids	类别对应的 MAPLE candidate IDs
prediction_time	当前推理时间
points_of_interest	network/camera/media/display 等场景标签
installed_apps	category 到 candidate ID 的映射
app_catalog	本窗口涉及的 app 元数据
observed_app_sequence	带时间戳的 app segments
timeline_windows	app + compressed eBPF 对齐窗口
system_evidence	全局 eBPF/system 摘要文本
compression	raw rows 到 compressed signals 的元数据
memory_pressure	normal/elevated/critical
available_scheduler_actions	bitmask 动作表

这些字段分成三类：第一类描述“接下来可

能打开什么 app”，如 app categories、app ids、installed apps；第二类描述“当前 OS 压力和证据”，如 scheduler evidence、memory pressure、system evidence；第三类描述“证据怎么来的”，如 observed app sequence、timeline windows、compression。这样模型既能预测用户行为，也能选择系统调度动作。

真实 Context JSON 摘录

```
{
  // 用户可见 app 预测候选, 不允许直接放 Binder/Memory
  "historical_app_categories": [
    "Media Codec",
    "Display Composition",
    "Network IO",
    "Communication",
    "Navigation/Location"
  ],
  // OS/eBPF 证据只作为推理和调度上下文
  "scheduler_evidence_categories": [
    "App Process Runtime",
    "Network IO",
    "Memory Management"
  ],
  "memory_pressure": "normal: no direct reclaim pressure was observed",
  // 证明模型输入来自 raw trace 的压缩摘要
  "compression": {
    "strategy": "windowed_count_aggregation",
    "raw_ebpf_rows_for_audit": 13733,
    "observed_app_sequence_rows": 6,
    "compressed_ebpf_signal_rows": 9
  }
}
```

注释强调数据边界：app 预测候选和 OS 调度证据分开。这样避免模型把 Memory/Binder 当成“下一个 app”，同时仍让这些系统信号影响 scheduler bits。

压缩不是丢证据，而是换表达

Raw trace

- 保存在 `/sdcard/MEMO/logs/*.trace`
- 用于 audit 和复现实验
- 可能上万行

Model input

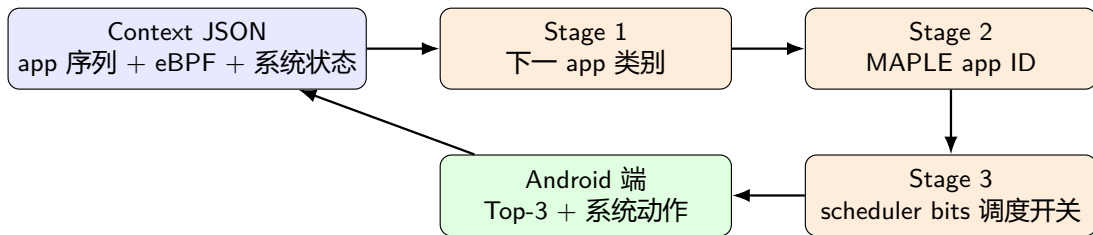
- `event_type`: 事件类型
- `detail`: 语义说明
- `count`: 重复次数
- `rate_per_sec`: 每秒频率
- `aligned app window`: 对应的 app 时间窗

repeated low-level events → compact semantic evidence 压缩的主要原因是 prompt 长度

和端侧推理成本。上万行 raw eBPF 里很多行只是在重复说明同一种系统活动，逐行输入会让小模型被噪声淹没；计数后保留“发生过、发生多少、在哪个 app 时间窗发生”，这对预测和调度更有用。

MAPLE 三阶段

Context JSON 是 MAPLE 每次推理读取的结构化输入：里面有 app 序列、压缩 eBPF、系统状态和可选调度动作表。



为什么两阶段预测

Stage 1 先缩小用户意图类别；Stage 2 只在该类别的 candidate IDs 内选择，降低小模型直接从复杂系统上下文生成具体 app 的难度。

Stage 1 prompt: 预测用户可见 app 类别

```
Pick the next user-facing Android app category
from real MEMO app history plus eBPF/OS context.
Answer exactly: <Category> (<percent>%).
Use only one category name from Candidates.
Do not answer Memory Management, Binder, service manager,
process runtime, or other OS-only labels as the app category.
Candidates: Media Codec, Display Composition, Network IO,
Communication, and Navigation/Location.
Memory is scheduler context, not an app category:
normal: no direct reclaim pressure was observed.
Prediction:
```

意图

eBPF/OS 证据仍进入 prompt, 但它们不再和 app category 竞争; Memory/Binder/process 是调度上下文, 不是用户要打开的 app。

Stage 1 真实输出

```
[3] MODEL RAW OUTPUT:  
"<Category> Media Codec (100%)"  
  
[4] PARSED RESULT:  
-> Predicted category: "Media Codec"  
    confidence: 100%
```

上半部分是 MAPLE 小模型原始输出，下半部分是 Android/C++ parser 解析后的结构化结果。Media Codec 表示模型判断用户下一步更可能进入媒体播放/音视频相关 app，而不是把 Memory Management、Binder 或进程运行时误当成 app。这个结果之后会进入 Stage 2，在媒体候选 ID 中选择具体 MAPLE App ID。

Source: docs/real_device_experiments/synthetic_user_30_2026_07_01/
rerun_maple_outputs/outputs/01_media_1_scenario_maple_output.txt

Stage 2 prompt: 类别到候选 App ID

```
Choose one MAPLE candidate ID for this eBPF category.  
Return one complete valid answer.  
Predicted category: Media Codec.  
Candidate IDs: 260.  
Recent IDs: 260, 115, 245, 110, and 270.  
Valid answers:  
- This user will use App 260.  
Prediction:  
  
MODEL RAW OUTPUT:  
"This user will use App 260."
```

说明

App 260 不是硬编码成某个包名；Android 端再通过本机安装应用扫描和 category 匹配映射成真实可启动 app。

Stage 3: scheduler_bits 动作协议

bit	action id	Android action
0	warm_launch_top1	预热 Top-1, 并回 HOME
1	warm_launch_top2_if_idle	空闲时预热 Top-2
2	trim_memory_low_priority_apps	对低优先级候选发 trim-memory
3	kill_selected_background_package	安全停止低优先级后台包
4	drop_cache_if_critical_memory	仅 critical memory 时 drop cache
5	refresh_network_stats	刷新 network stats
6	refresh_service_manager	刷新 service manager context
7	reduce_prewarm_when_display_busy	UI/display 忙时降低预热强度
8	skip_camera_prewarm_when_thermal_high	高温时跳过 camera/media 预热

真实输出示例

`scheduler_bits = 101010100`: 执行 bit0、bit2、bit4、bit6 对应动作; 具体是否落地还会经过内存/温度/前台安全检查。

小模型输出解析也要工程化

真实遇到的输出

```
"<Category>Communication (<10%))"
```

这不是理想格式，但端侧小模型会这样输出。

parser 修正

- 允许 <Category> prefix。
- 允许 <10%。
- 允许多余右括号。
- App ID 110 在 app prediction 语境下映射为 Communication。

对应代码

```
llm/maple/maple_engine/src/result_parser.cpp
```

从 MAPLE App ID 到真实 Top-3：映射表怎么来

核心思想

MAPLE 输出的是“语义候选”（例如 Communication / App 110），不是固定包名。Android 端每次从本机实际安装 app 里自动找能匹配这个语义的真实应用。

- ① 用 PackageManager（Android 的系统应用扫描接口）扫描所有 launcher app：拿到包名、应用名、图标、launch intent。
- ② 为每个 app 自动打标签：默认浏览器/SMS/电话角色、权限、intent 能力、ApplicationInfo.category、包名和 label 关键词。
- ③ 把这些标签合并成语义类别：Communication、Network IO、Camera Service、Media Codec、Payment/Security 等。
- ④ 对 MAPLE 输出的类别/App ID 建候选池：只保留本机真实可启动 app，然后按匹配分排序。

排序大致是：默认角色/系统 intent 能力 > 权限和 Android category > 包名/label 关键词 > 最近使用和用户安装权重。所以这里没有写死“Communication = QQ”。QQ 只是这台手机上被扫描出来且匹配 Communication 的一个候选；换一台手机时，候选池会重新由那台手机的已安装应用生成。

从 MAPLE App ID 到真实 Top-3：用户看到什么

同一个 MAPLE 语义在不同手机上的落点

- 手机 A 装了 QQ：Communication → QQ / Messages / Phone
- 手机 B 没装 QQ 但装了微信：Communication → WeChat / Messages / Phone
- 只有系统应用：Communication → Messages / Phone / Contacts

用户界面的约束

- Top-3 必须有真实包名和 launch intent。
- 能显示 app label 和图标。
- 点击后能打开对应应用。
- Binder、SurfaceFlinger、Memory Management 只能作为调度证据，不会显示成 app。

“自然落到微信”不是模型认识微信，而是扫描器看到微信的包名、label、权限、intent 能力和通信类特征后，把它放进 Communication 候选池。MAPLE 只负责说“接下来像 Communication”，最后显示哪一个 app 由本机扫描结果决定。

ActionExecutor 执行的是真动作

```
# warm launch Top-1
am start -W -n <component>
sleep 1
am start -a android.intent.action.MAIN \
  -c android.intent.category.HOME

# memory trim
am send-trim-memory <package> RUNNING_LOW
```

```
# network context refresh
dumpsys netstats

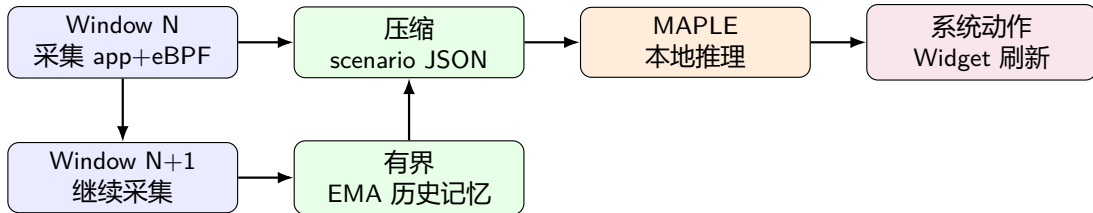
# service context refresh
service list

# critical memory only
sync; echo 1 > /proc/sys/vm/drop_caches
```

安全边界

不杀系统包，不杀前台 app，不杀推荐候选；高内存/高温压力下减少预热而不是盲目启动更多 app。

实时模式任务队列



线程控制

一个 collection flow + 一个 MAPLE worker; 推理慢也不停止下一窗口采集, 历史 memory 有长度上限, 避免后台线程和上下文无限膨胀。

Widget：把预测变成可用的产品入口

为什么需要 Widget

- MEMO 不是让人一直打开主 App 看日志。
- 实时模式每个窗口更新 Top-3 后，Widget 在桌面展示真实应用图标。
- 用户可以直接点推荐应用，不需要理解 eBPF、Binder、scheduler bits。

Widget 显示什么

- Top-3 真实 app 图标和名称。
- 最新更新时间，说明推荐是否来自最近窗口。
- 点击图标直接打开对应应用。

Widget 是用户层面的核心出口：eBPF 和 MAPLE 在后台工作，用户只看到“接下来可能要用的三个 app”。这让系统预测变成可点击的手机体验，而不是只停留在实验报告里。

实验总览：我们要回答什么

问题 1：实时产品闭环能不能连续运行？

对应 9 分钟实时 Top-3 变化实验。看三个连续窗口、Top-3 更新时间、MAPLE 耗时、scheduler bits。

问题 2：eBPF/OS 证据是否真的帮助预测？

对应 30 条真实记录预测实验和理论消融。看 Top-1、Top-3 Hit、Precision/Recall/F1、MRR。

问题 3：模型输出能不能进入系统调度？

对应 9 分钟调度记录和样本动作记录。看 warm launch、memory trim、service refresh，以及哪些动作被安全规则跳过。

实验之间的关系

- 9 分钟实验验证产品形态：连续采集、连续推理、连续刷新推荐。
- 30 条预测实验验证预测效果：同一批样本上比较 app-only 和 app+OS/eBPF。
- 理论消融验证信号价值：在有明确 ground truth 的合成数据上系统删去不同证据。
- 调度记录验证执行闭环：MAPLE 输出不止文本，而是真的触发 Android 动作。

这几组实验不是重复展示同一件事，而是分别回答“能不能运行”“预测有没有变好”“eBPF 是否有独立贡献”“模型能不能控制系统动作”。

9 分钟实时实验：动机与方案

为什么做 9 分钟

实时模式的产品设定是每 3 分钟形成一个窗口，持续采集 app 序列、eBPF 和系统状态，然后异步 MAPLE 推理并刷新 Top-3 和调度动作。9 分钟正好覆盖 3 个完整窗口，可以观察推荐是否随场景变化。

窗口 1

network/browser：浏览、网络收发、页面渲染。

窗口 2

camera/photo：相机、图片、显示与文件访问。

窗口 3

media/communication：媒体、通信、服务刷新。

每个窗口都在手机端真实启动已安装 app，并自然产生 eBPF 证据；不是手写 JSON，也不是 host Python 伪造 trace。Host 只负责触发和拉取结果。

9 分钟实时实验：结果

Phase	raw eBPF	app segments	MAPLE time	Top-3
network	19263	52	29106 ms	Messages / Camera / Chrome
camera	19289	55	37196 ms	QQ / Messages / Camera
media/comm	19260	54	23839 ms	Messages / Camera / QQ

结果解读

- 三个窗口都采到约 1.9 万条 raw eBPF 事件，说明采集链路持续工作。
- app segments 在 52-55 之间，说明实验里确实存在多次真实前台 app 变化。
- Top-3 在窗口间发生变化，说明推荐不是固定列表，而是随最近 app + OS 证据刷新。

Source: docs/real_device_experiments/realtime_top3_shift_2026_07_01/

9 分钟实验中的调度记录

```
phase 1 scheduler_bits = 100000000
-> maple_warm_launch_top1: Messages

phase 2 scheduler_bits = 100000000
-> maple_warm_launch_top1: QQ

phase 3 scheduler_bits = 101010100
-> maple_warm_launch_top1: Messages
-> maple_trim_memory: Camera, QQ
-> maple_drop_cache: skipped, memory not critical
-> maple_service_manager_refresh: ok
```

`scheduler_bits` 是 MAPLE 输出的动作开关。第 1、2 个窗口只触发 bit0，也就是预热 Top-1。第 3 个窗口输出 101010100，同时触发 Top-1 预热、低优先级候选 memory trim、service manager refresh；drop cache 被安全规则拦截，因为当时内存不是 critical。

Source: docs/real_device_experiments/realtime_top3_shift_2026_07_01/

30 条真实记录预测实验：实验设计

样本如何来

- 设定 3 个 persona: media、productivity、social。
- Codex 根据 persona 生成 10 条短 app 使用脚本；脚本规定“已观察到的 app 序列”和“被隐藏的未来 Top-3”。
- Android runner 驱动真实点击/启动已安装 app。
- 每条样本都重新采集真实 app timeline 和 raw eBPF。

如何形成 ground truth

- 每条样本有输入 app 序列和未来 Top-3。
- 未来 Top-3 在 scenario 中被 mask。
- MAPLE 只能看输入序列 + eBPF/OS context。
- 预测完成后再用隐藏 truth 评分。

这里实事求是：使用意图脚本和 ground truth 是 Codex 按三类 persona 生成的；但执行阶段仍然是在 Android 设备上真实启动 app、发 input 命令、自然产生 raw eBPF。实验比较 app-only baseline 和 MEMO app+OS/eBPF，用来判断 OS 证据是否让预测更好。

30 条预测实验：评估指标

令预测集合为 P_3 , ground truth 集合为 T_3 , 预测排序为 p_1, p_2, p_3 。

集合类指标

- Top-1 Acc: $1[p_1 \in T_3]$
- Top-3 Hit: $1[P_3 \cap T_3 \neq \emptyset]$
- Precision@3: $|P_3 \cap T_3|/3$
- Recall@3: $|P_3 \cap T_3|/|T_3|$
- F1@3: $2PR/(P + R)$

排序类指标

MRR: 如果第一个命中的 truth 出现在第 r 位, 则得分 $1/r$; 如果没有命中则为 0。

代码如何算

Kotlin runner 对每条样本保存 `full_memo_top3`、`app_only_top3` 和 `ground_truth_top3`, 再逐条求交集、命中位置和平均值。

30 条预测实验：单条指标怎么算

Predicted Top-3:

QQ / Tencent Meeting / Chrome

Ground Truth Top-3:

QQ / Chrome / Camera

Intersection size = 2 (QQ, Chrome)

Top-1 Acc = 1, because QQ is in truth

Top-3 Hit = 1, because intersection is not empty

Precision@3 = $2 / 3 = 0.667$

Recall@3 = $2 / 3 = 0.667$

F1@3 = 0.667

MRR = $1 / \text{rank}(\text{QQ}) = 1.0$

这个例子有命中但不完全一致：QQ 和 Chrome 命中，Tencent Meeting 是误报，Camera 是漏报。所以 Top-3 Hit 仍然是 1，但 Precision、Recall、F1 都是 2/3。MRR 只看第一个命中的 truth 在预测列表中的位置；因为 QQ 排第一，所以 MRR 是 1。

理论消融：先在可控数据上验证 eBPF 信号

实验设置

- Chengyu Fan 的理论/合成消融实验。
- 1000 条带 ground truth 的合成样本。
- 同一个 MAPLE PromptBuilder, 逐组删除 app sequence、Binder、file、memory 等证据。
- 目的：先回答“eBPF/OS 证据本身是否有用”。

Config	DeepSeek	Qwen CPU
Full eBPF	88.8%	63.0%
Binder + File	91.0%	52.8%
App sequence only	51.1%	30.1%
Random	5.6%	4.8%

和后面真实实验的一致性

理论实验在可控 ground truth 上显示：多源 eBPF/OS 证据显著强于仅 app 序列。后面的真实设备最终实验虽然样本更小、路径更接近产品运行，也得到同方向结果：**MEMO app+OS/eBPF** 在 Top-3 Hit、F1@3、MRR 上均超过 app-only baseline。

Source: docs/theoretical_ablation_experiment/synthetic_ablation_report.md

30 条真实记录：最终预测结果

Config	Top-1 Acc	Top-3 Hit	Precision@3	Recall@3	F1@3	MRR
App-only baseline	20.00%	46.67%	47.78%	47.78%	47.78%	31.11%
MEMO app+OS/eBPF	23.33%	50.00%	52.22%	52.22%	52.22%	34.44%

相对 app-only 提升

- Top-3 Hit: +3.33 pct points
- F1@3: +4.44 pct points
- Top-1 Accuracy: +3.33 pct points

结论

full path 提升不大，但方向一致：加入 eBPF/OS context 后，Top-3 Hit、F1@3、MRR 都高于只看 app 序列。

Source: docs/real_device_experiments/synthetic_user_30_2026_07_01/rerun_inference_only_report.json

单条样本的直观结果

```
sample_id: media_2
persona: media_browsing

MAPLE predicted category:
Communication

Predicted Top-3:
QQ / Tencent Meeting / Chrome

Ground Truth Top-3:
QQ / Tencent Meeting / Chrome

metrics:
hit_top1 = true
hit_top3 = true
F1@3 = 1.0
exact_order_top3 = true
```

这是一个 media_browsing 风格样本：前半段脚本让设备在媒体/浏览场景中切换，真实采集里会出现网络收发、Binder、显示刷新和 app 切换证据。后半段被 mask 的未来动作是 QQ、Tencent Meeting、Chrome，语义上都和“媒体浏览后继续通信/会议/网页”相邻。Full path 结合 app 序列和 OS/eBPF 证据后把三个都排出来，所以这条样本全对。

系统调度性能实验：Stage 3 是否有用

实验设计

先用真实 eBPF + MAPLE 选择同一个真实 Top-1 app: QQ。Baseline 对 QQ 做冷启动；Stage 3 先执行同一组 scheduler_bits=101010100，再启动同一个 QQ。因此比较的是“同一目标 app 无调度”与“同一目标 app 经 MAPLE 调度后”的系统指标。

Metric	No scheduler	Stage 3	Improvement
Launch TotalTime	433 ms	168 ms	61.20%
WaitTime	440 ms	173 ms	60.68%
综合压力分数	422.56	9.71	97.70%
CPU busy	26.70%	20.79%	22.13%
iowait	0.094%	0.023%	75.42%
reclaim	10046	0	100.00%
jank rate	1.240%	0.886%	28.56%

本页只评价系统调度性能，不评价预测准确率。正数表示 lower-is-better

指标下降。Stage 3 真实执行了 7 条动作记录，其中包括 Top-1 warm launch、memory trim、service manager refresh；drop cache 被安全规则跳过，因为当时内存不是 critical。

Source: docs/real_device_experiments/system_scheduler_performance/latest_pressure_experiment.json

关键代码：三分钟采集窗口

`RealtimeWindowRunner.kt`

这个文件负责在手机内完成一个窗口的真实采集：启动 raw eBPF collector，同时采样前台 app，最后输出 app timeline、raw trace 和 timeline windows。

为什么关键

它决定了“输入是不是来自真实使用”。如果这里仍然依赖 host Python 或手写 JSON，后面的 MAPLE 推理和调度都不能算端侧产品闭环。

Source: `app/src/main/java/com/memoos/realtime/RealtimeWindowRunner.kt`

关键代码：有界实时记忆

`RealtimeMemoryStore.kt`

这个文件维护历史 app 类别和资源压力的 EMA，也就是指数衰减平均。它只保留固定长度的历史窗口，不让后台 memory 越跑越大。

为什么关键

实时模式不能只看最近三分钟，也不能无限保存所有历史。EMA 让模型知道用户近期偏好，同时控制端侧开销。

Source: `app/src/main/java/com/memoos/realtime/RealtimeMemoryStore.kt`

关键代码：MAPLE 输入构造

`MapleScenarioBuilder.kt`

这个文件把 app 序列、eBPF counter、系统状态、安装应用表、scheduler 动作表组合成 MAPLE 使用的 scenario JSON。

为什么关键

它是“app + OS 到 app + OS”的数据边界：既要把 app 预测候选给模型，也要把 OS 证据作为调度上下文给模型。

Source: `app/src/main/java/com/memoos/maple/MapleScenarioBuilder.kt`

关键代码：本地模型调用

MapleShellBackend.kt

这个文件在手机本地调用 MAPLE/llama.cpp 可执行文件，传入 scenario JSON，拿回 Top-3 预测、pressure analysis 和 scheduler bits。

为什么关键

它保证预测不是电脑上跑脚本，而是 Android 设备侧本地推理；拔掉数据线后仍然可以通过 App/Widget 运行。

Source: `app/src/main/java/com/memoos/maple/MapleShellBackend.kt`

关键代码：真实应用映射

AppIdMapping.kt

这个文件扫描本机安装 app，把 MAPLE 的类别/App ID 映射成可启动的 QQ、Chrome、Camera 等真实应用。

为什么关键

用户看到的是应用，不是进程、Binder 或 SurfaceFlinger。这个映射层把模型输出变成用户能理解、能点击的 Top-3。

Source: `app/src/main/java/com/memoos/action/AppIdMapping.kt`

关键代码：系统调度动作

ActionExecutor.kt

这个文件解释 scheduler bits, 执行 warm launch、memory trim、network/service refresh, 并做内存和温度安全检查。

为什么关键

MAPLE 输出不能只停留在“建议”。ActionExecutor 把模型 bitmask 变成可执行 Android 动作, 同时避免杀前台 app、系统包或推荐候选。

Source: `app/src/main/java/com/memoos/action/ActionExecutor.kt`

关键代码：桌面 Widget

`MemoWidgetProvider.kt`

这个文件在桌面展示 Top-3 真实应用和最新更新时间，点击图标可以直接打开对应应用。

为什么关键

Widget 是用户真正会用到的入口。后台的 eBPF、MAPLE 和调度动作不需要暴露给用户；用户只需要看到“接下来可能要打开的三个 app”。

Source: `app/src/main/java/com/memoos/widget/MemoWidgetProvider.kt`

关键代码：提示词与输出解析

`prompt_builder.cpp` / `result_parser.cpp`

这两个文件约束小模型输出 app 类别、App ID 和 scheduler bits，并解析不规整的模型文本。

为什么关键

端侧小模型不够“听话”，输出可能带多余括号或解释文字。parser 必须把这些脏输出尽量转成结构化结果，才能驱动推荐和调度。

Source: `llm/maple/maple_engine/src/prompt_builder.cpp` and `llm/maple/maple_engine/src/result_parser.cpp`

最终系统做到了什么

端侧闭环



- app+OS 共同进入模型，不再只是浅层 app sequence (应用序列)。
- 模型输出不止推荐，也控制预定义系统动作。
- Widget 让 Top-3 推荐成为桌面可点击入口，方便用户直接打开预测应用。
- 真实 Pixel 5 上完成 eBPF 采集、推理、调度、Widget 展示和实验归档。

限制与下一步

当前限制

- 小模型能力有限，不够“听话”：即使 prompt 要求固定格式，它也可能多输出括号、解释或错误字段。
- 目前已经有 parser 处理脏输出，但还不够 robust（鲁棒）：需要覆盖更多奇怪输出。
- 实时性的代价是后台常驻：系统持续采集，每三分钟推理一次；实时性和轻量性是一对 tradeoff。

下一步

- 更细粒度 scheduler policy：把 bitmask 从粗粒度动作扩展成带强度、时机和安全阈值的调度策略。
- 强化 app mapping：更自动地识别新安装 app 的语义类别，减少依赖包名关键词。
- 完善 capability report 和部署脚本：让设备端清楚报告 root、BTF、tracefs、模型文件和 collector 状态。

Thanks

MEMO-Appflow: app + OS evidence $\rightarrow$ local AI reasoning $\rightarrow$ app + OS actions